CLASE 2 · MÓDULO 2

Principios de las redes neuronales

De la neurona de McCulloch y Pitts al perceptrón

github.com/datahuba/curso-redes-neuronales · Basado en el curso del Dr. J. Irving Vasquez (jivg.org)

Agenda de hoy

Tiempo	Tema
0:00	Repaso de la Clase 1
0:05	Modelos paramétricos
0:15	Neurona de McCulloch y Pitts
0:40	Práctica: McCulloch y Pitts
1:00	Descanso
1:10	El perceptrón
1:30	Regla de aprendizaje
1:50	Práctica: perceptrón en Python
2:20	Cierre, quiz y tarea

Al terminar la clase podrás…

1

Calcular a mano

la salida de una neurona de McCulloch y Pitts y de un perceptrón.

2

Aplicar la regla

de aprendizaje de Rosenblatt, paso a paso.

3

Explicar XOR

por qué un perceptrón no puede aprenderlo.

4

Programar

ambas neuronas en Python.

Repaso de la Clase 1

¿Aprendizaje supervisado?

Aprender de pares (entrada x, etiqueta y).

¿Generalizar?

Acertar en ejemplos que nunca vio.

Hoy: la primera pieza de una red neuronal, la neurona artificial.

Modelos paramétricos

$$\hat{y} = \phi_\theta(X)$$

Símbolo	Significa
X	La entrada (los datos)
θ	Los parámetros: las perillas
$\hat{y}$	La predicción del modelo
y	La respuesta correcta

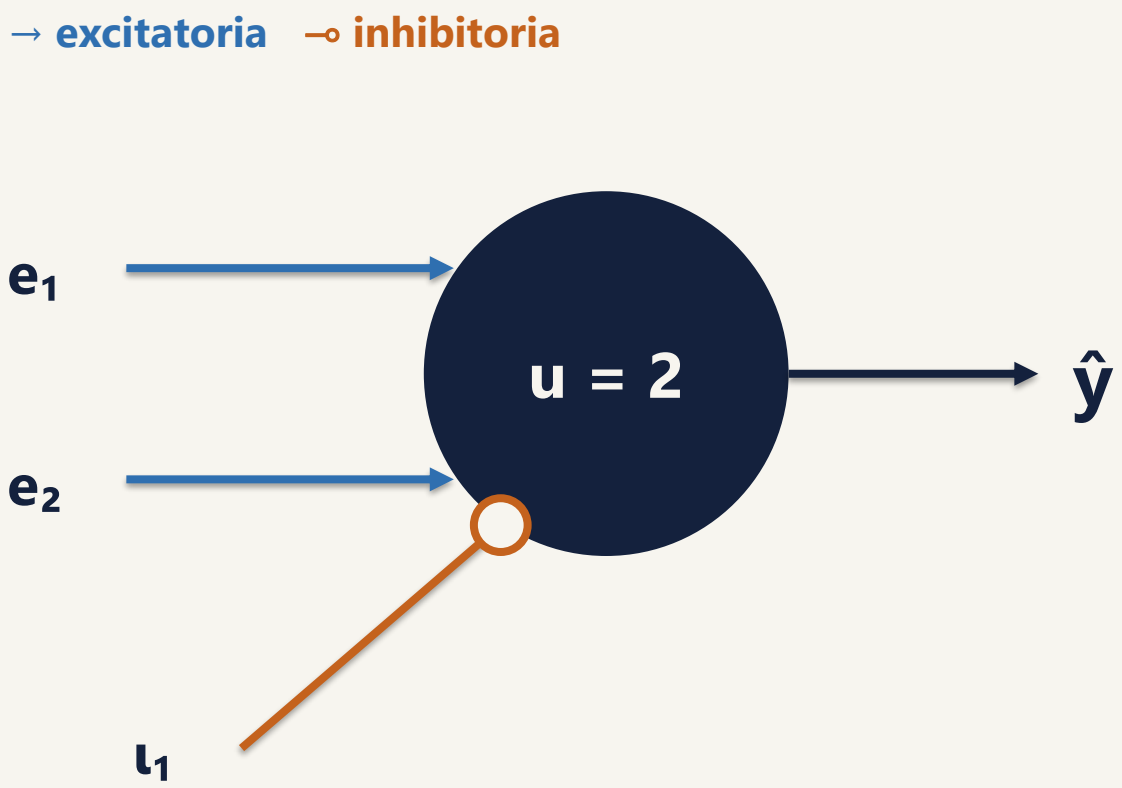
IDEA CENTRAL DEL CURSO

Aprender = encontrar θ tal que $\hat{y} \approx y$

Ejemplo: la recta $\hat{y} = w \cdot x + b$

En redes: θ = pesos W y sesgo b

Neurona de McCulloch y Pitts (1943)



Elemento	Valores
E: excitatorias	0 o 1
I: inhibitorias	0 o 1
u: umbral	0, 1, 2, …
ŷ: salida	0 o 1

La primera neurona artificial: una simplificación de la neurona biológica.

Cómo decide la neurona

1. **Alguna inhibitoria vale 1** → $\hat{y} = 0$
2. **Si no, $\Sigma e_i \geq u$** → $\hat{y} = 1$
3. **En cualquier otro caso** → $\hat{y} = 0$

Las reglas se aplican en este orden.

ANALOGÍA

Un portero de discoteca

Inhibitorias = lista negra: si estás, no entras.

Si no, entras con al menos u invitaciones (excitatorias).

Compuertas lógicas con McCulloch y Pitts

e_1	e_2	Σ	AND (u=2)	OR (u=1)
0	0	0	0	0
0	1	1	0	1
1	0	1	0	1
1	1	2	1	1

Compuerta	Configuración
AND	u = n
OR	u = 1
≥ k de n	u = k
NOT	E = ∅, 1 inhibitoria, u = 0

NOT: sin excitatorias la suma es 0 ≥ u = 0, así que se activa… salvo que la inhibitoria valga 1.

Limitaciones y ejercicios

Le falta…

- **Pesos:** todas las entradas valen igual.
- **Valores reales:** solo acepta 0 o 1.
- **Aprender:** el umbral se fija a mano.

En papel · 5 min

1. Umbral para “al menos 2 de 3”.
2. Tabla de $C(E=\{e_1,e_2\}, I=\emptyset, u=2)$.
3. $C(E=\{e_1,e_2\}, I=\{\iota_1\}, u=1)$ con $\iota_1 = 1$.

La limitación más importante: no aprende. Eso lo resuelve el perceptrón →

20 MIN · 3 PUNTOS

Práctica · notebooks/1_01_mcculloch_pitts

TODO 1 · 2 pts neuronaMyP(E, I, u) sin NumPy

TODO 2 · 1 pt leer E, I y u arbitrarios

Traduce las 3 reglas al código, en el mismo orden.

```
# comprobaciones
neuronaMyP([], [0], 0)     # 1
neuronaMyP([], [1], 0)     # 0
neuronaMyP([1, 1], [], 2)  # 1
neuronaMyP([1, 0], [], 2)  # 0
neuronaMyP([1], [0], 1)    # 1
```

Sin Python: abre el notebook en Google Colab.

Descanso · 10 min

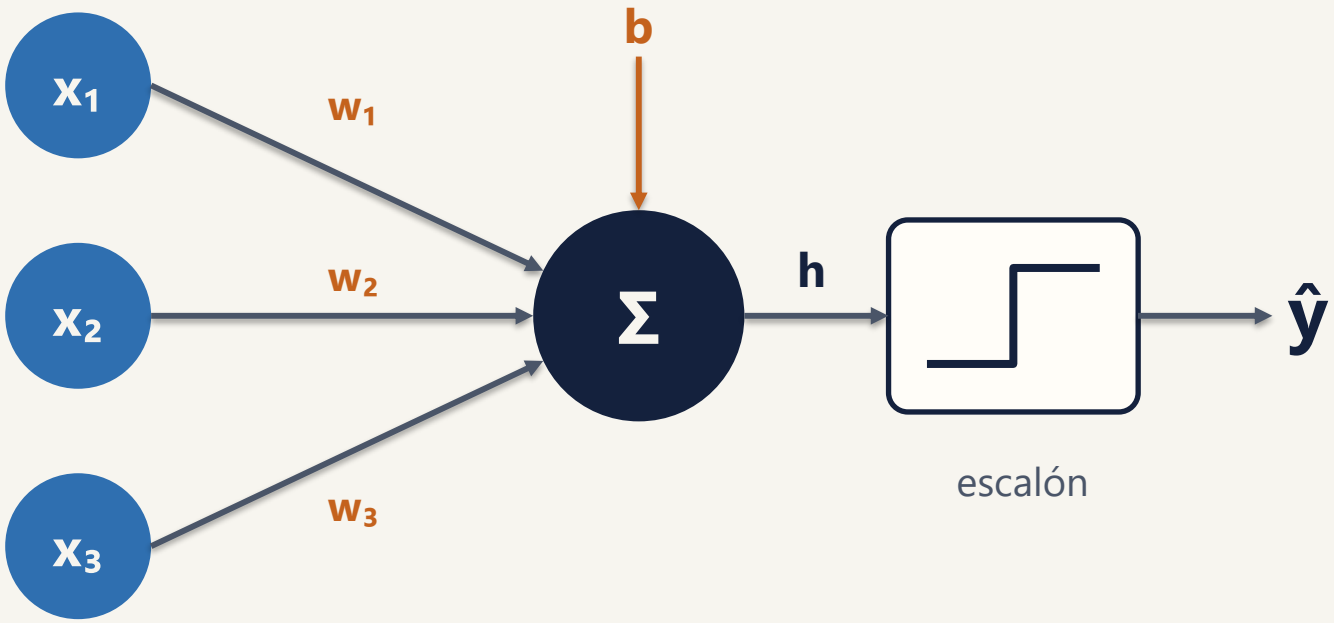
Después: el perceptrón

El perceptrón (Rosenblatt, 1958)

	McCulloch y Pitts	Perceptrón
Entradas	Solo 0 o 1	Números reales
Importancia	Todas iguales	Un peso w_i por entrada
Desplazamiento	Umbral u fijo	Sesgo b ajustable
¿Aprende?	No	**Sí**

El primero fue una máquina física con fotoceldas como “ojos” que ajustaba sus pesos al equivocarse.

El modelo en dos pasos



Truco: una entrada extra $x_0 = 1$ con peso b; así el sesgo es "un peso más".

1 · Combinación lineal

$$h = \Sigma\, w_i \cdot x_i + b$$

2 · Activación escalón

$$\hat{y} = 1 \text{ si } h \geq 0$$

$$\hat{y} = 0 \text{ si } h < 0$$

Ejemplos: calcular la salida

Del libro

$W = [0.7, -0.3, 0.4] \cdot b = 0$

$X = [8, 5, 6]$

$h = 0.7 \cdot 8 - 0.3 \cdot 5 + 0.4 \cdot 6$

$= 5.6 - 1.5 + 2.4 = 6.5$

$\hat{y} = f(6.5) = 1$

¿Salgo a correr?

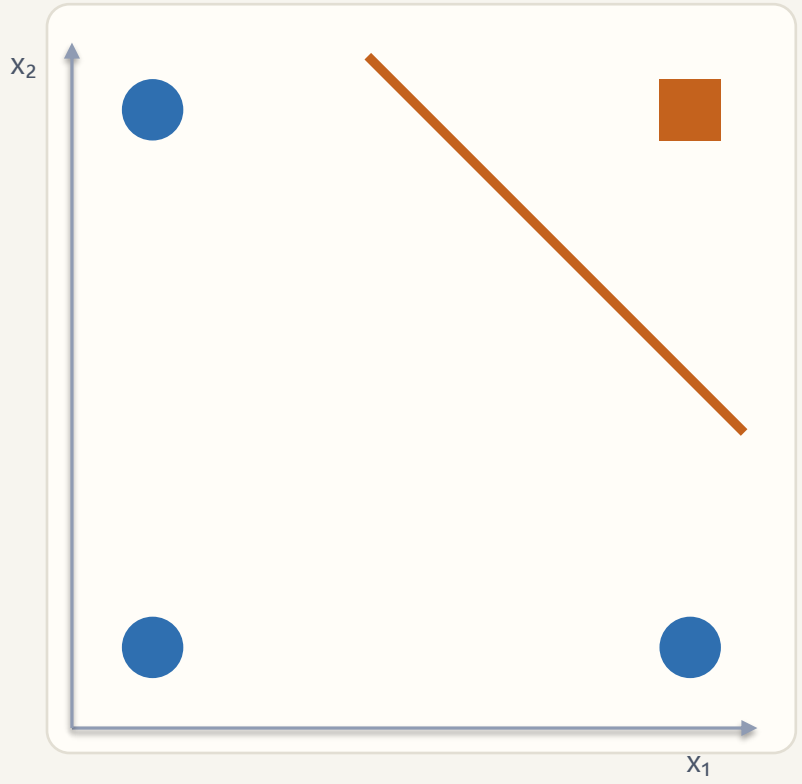
x_1 = ¿hace sol? · x_2 = ¿estoy cansado?

$w = [2, -3] \cdot b = -1$

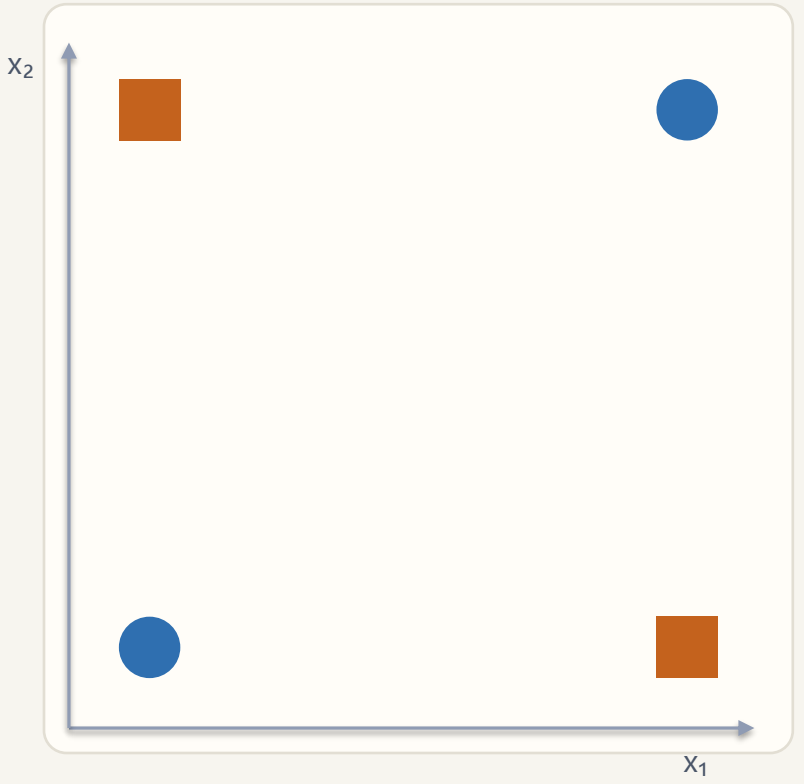
Sol y descansado: $h = 1 \rightarrow$ **salgo**

Sol y cansado: $h = -2 \rightarrow$ **no salgo**

El perceptrón traza una recta



AND: separable



XOR: ¿una recta?

■ clase 1 ● clase 0

Frontera de decisión

$w_1x_1 + w_2x_2 + b = 0$

AND: w = [1, 1], b = −1.5

OR: w = [1, 1], b = −0.5

XOR: ninguna recta separa los cuadrados de los círculos.

→ no es linealmente separable.

Regla de aprendizaje de Rosenblatt

$$W \leftarrow W + \eta\,(y - \hat{y})\,X$$
$$b \leftarrow b + \eta\,(y - \hat{y})$$

Caso	Error $y - \hat{y}$	Efecto
Acertó	0	No cambia nada
Dijo 0, debía decir 1	+1	Suma η·X: h sube
Dijo 1, debía decir 0	−1	Resta η·X: h baja

η = tasa de aprendizaje · **época** = una pasada por todo el dataset · converge si los datos son linealmente separables

Ejemplo: un paso de aprendizaje

$X = [3, 5, 8] \cdot y = 0 \cdot \eta = 0.5 \cdot W^0 = [0.3, -0.4, 0.5] \cdot b^0 = 0.2$

#	Cálculo
1	$h = 0.9 - 2.0 + 4.0 + 0.2 = 3.1 \rightarrow \hat{y} = 1$
2	error $= 0 - 1 = -1$ (se equivocó)
3	$W^1 = W^0 - 0.5 \cdot [3, 5, 8] = [-1.2, -2.9, -3.5]$
4	$b^1 = 0.2 - 0.5 = -0.3$
5	Comprobar: $h = -46.4 < 0 \rightarrow \hat{y} = 0 = y$ ✓

Ejercicios en papel · 10 min

A

$w = [0.5, -0.3]$

$b = 0.4 \cdot \eta = 0.1$

$X = [1, 2] \cdot y = 0$

¿ŷ? ¿error? ¿nuevos pesos?

B

50 épocas sobre 27 ejemplos.

¿Cuántas presentaciones son repeticiones?

C

NAND vale 0 solo en (1, 1).

Encuentra pesos y sesgo.

DOCS/PRACTICA_02.PY

El perceptrón en Python

```
def predecir(x, w, b):
    h = sum(wi * xi for wi, xi in zip(w, x)) + b
    return 1 if h >= 0 else 0

def entrenar(X, y, eta=0.1, epocas=100):
    w, b = [0.0] * len(X[0]), 0.0
    for _ in range(epocas):
        for xi, yi in zip(X, y):
            error = yi - predecir(xi, w, b)
            w = [wj + eta*error*xj for wj, xj in zip(w, xi)]
            b += eta * error
    return w, b
```

EJECUTA

python docs/practica_02.py

AND, OR: aprende en 4 épocas

XOR: convergió=False tras 100 épocas

30 MIN

Práctica · notebooks del perceptrón

Notebook	Qué hacer	Pts
1_02_perceptron_AND	Ajustar pesos a mano para AND	—
1_03_1_perceptron_simple	Combinación lineal sin np.dot, escalón, pesos de OR	3
1_03_2_regla_de_aprendizaje	Inicializar, pérdida SAE, regla, hiperparámetros	4

Comprueba 1_03_1: inputs [0.7, −0.3], weights [0.1, 0.8], bias −0.1 → h = −0.27 → salida 0

En 1_03_2: sube las épocas a 20–50 y mira cómo baja la pérdida.

Para llevar

1. Neurona = suma ponderada + activación.
2. El perceptrón aprende corrigiendo pesos con el error.
3. Solo separa clases con una recta.

¿Y XOR? Conectando neuronas en capas. Próximas clases.

Tarea: notebooks 1_01, 1_03_1 y 1_03_2 (10 pts)

Leer: libro, “Neurona artificial moderna”